



Aprendizaje automático clásico

Sesión 2 · Del problema al primer modelo

Presenta:

Dr. Ulises Olivares Pinto



THINK **BIG** SPACE



HPC
DATA LAB

Meta de la sesión

1

Formular

Traducir una necesidad a X , y , decisión y riesgo

2

Explorar

Auditar, limpiar y comprender los datos antes de modelar

3

Defender

Comparar modelos y elegir una métrica según el costo del error

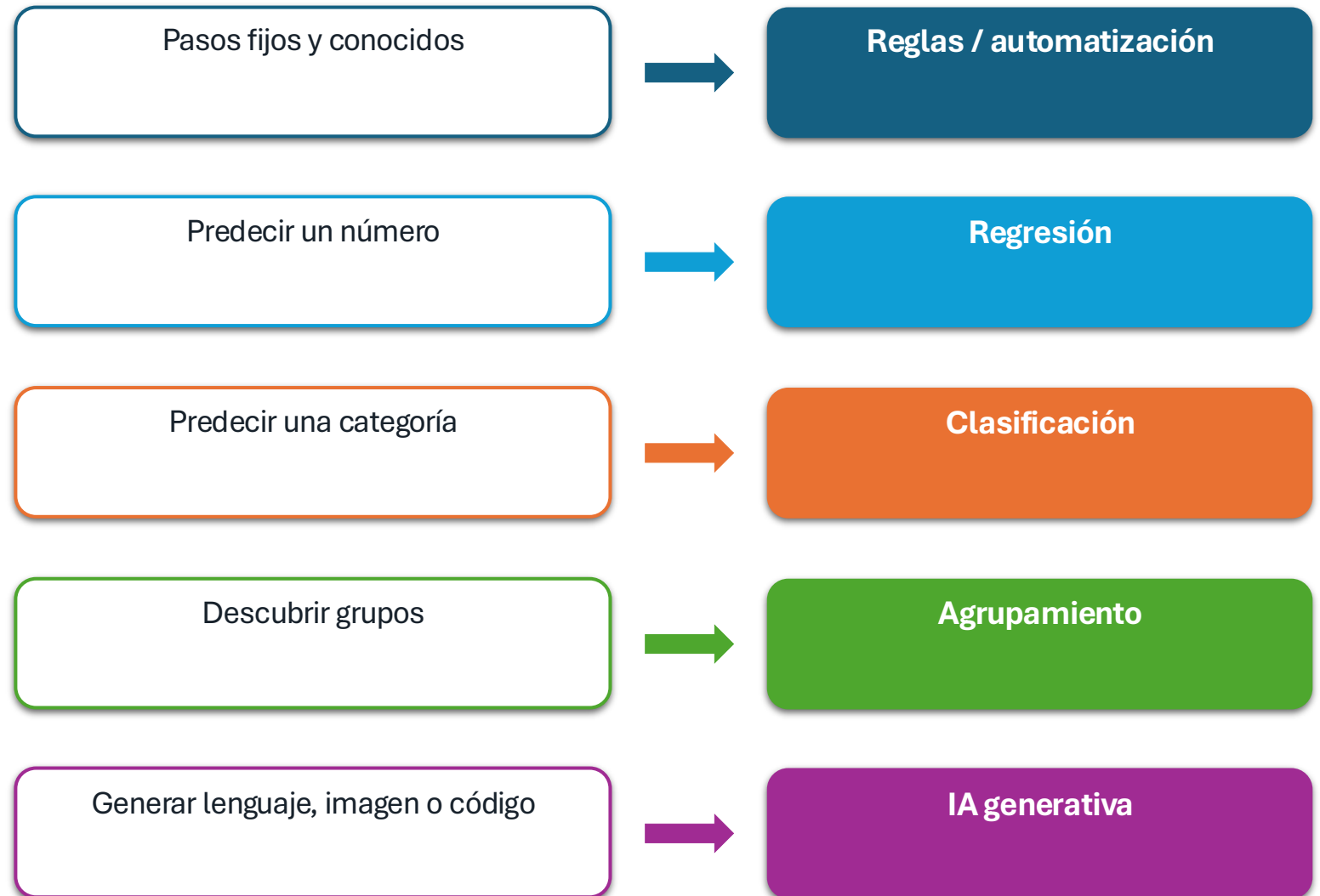
Reto inicial

¿Qué usarías primero?

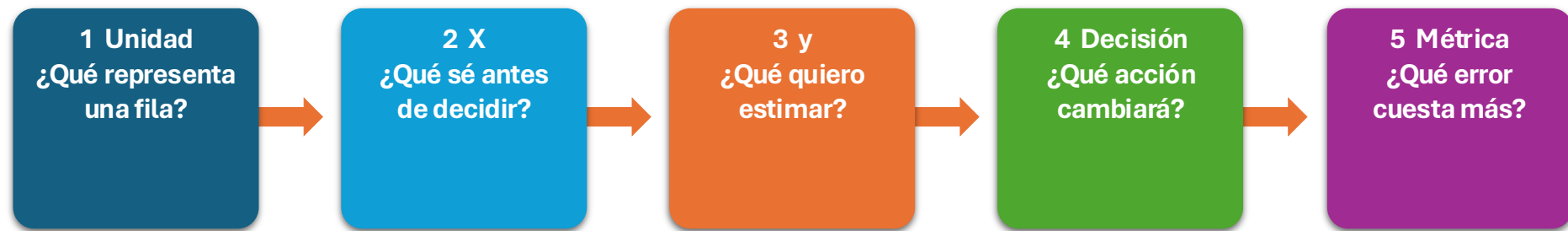
- 1 Enviar un reporte cada viernes
- 2 Estimar ventas de la próxima semana
- 3 Detectar una solicitud con posible fraude
- 4 Descubrir perfiles de consumo semejantes
- 5 Redactar una respuesta inicial a un cliente

Reglas · regresión · clasificación · agrupamiento · IA generativa

¿Qué usar primero?



Todo modelo comienza con una pregunta bien formulada



Ejemplo

Pedido → proveedor y distancia → días de entrega → priorizar seguimiento → MAE + retrasos críticos

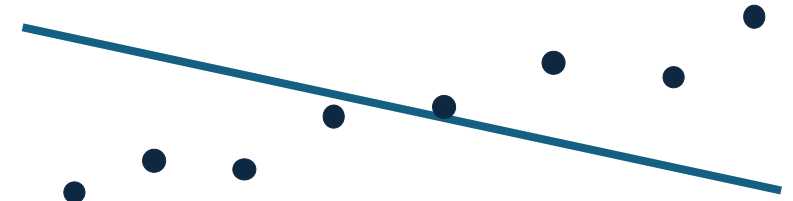
El modelo estima; una persona o un proceso autorizado decide.

Tres familias de problemas

REGRESIÓN

¿Cuánto?

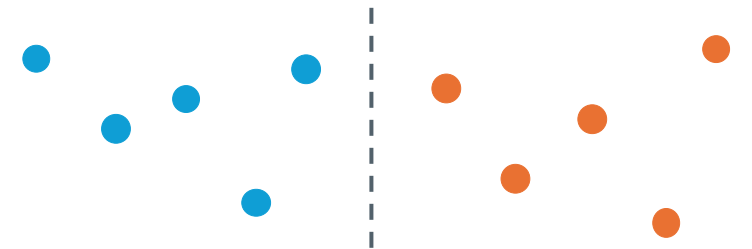
demanda · monto · tiempo



CLASIFICACIÓN

¿Qué categoría?

fraude/no fraude · riesgo



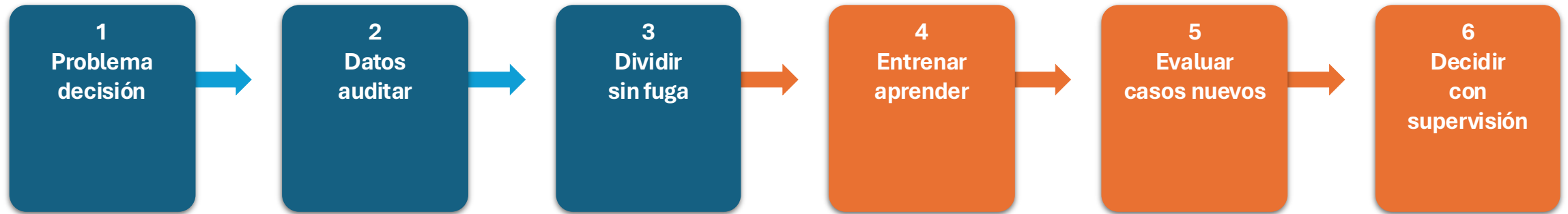
AGRUPAMIENTO

¿Qué casos se parecen?

segmentos · perfiles



El flujo completo cabe en seis decisiones



Un modelo útil no termina al entrenarse: se revisa cuando cambian los datos, la decisión o el costo del error.

Auditar antes de limpiar

Cinco preguntas convierten una tabla en evidencia confiable

ESTRUCTURA

¿Una fila = un cliente/contacto? ¿Los tipos son correctos?

FALTANTES

¿Cuánto falta y por qué? No imputar todavía.

DUPLICADOS

¿Misma observación o eventos realmente repetidos?

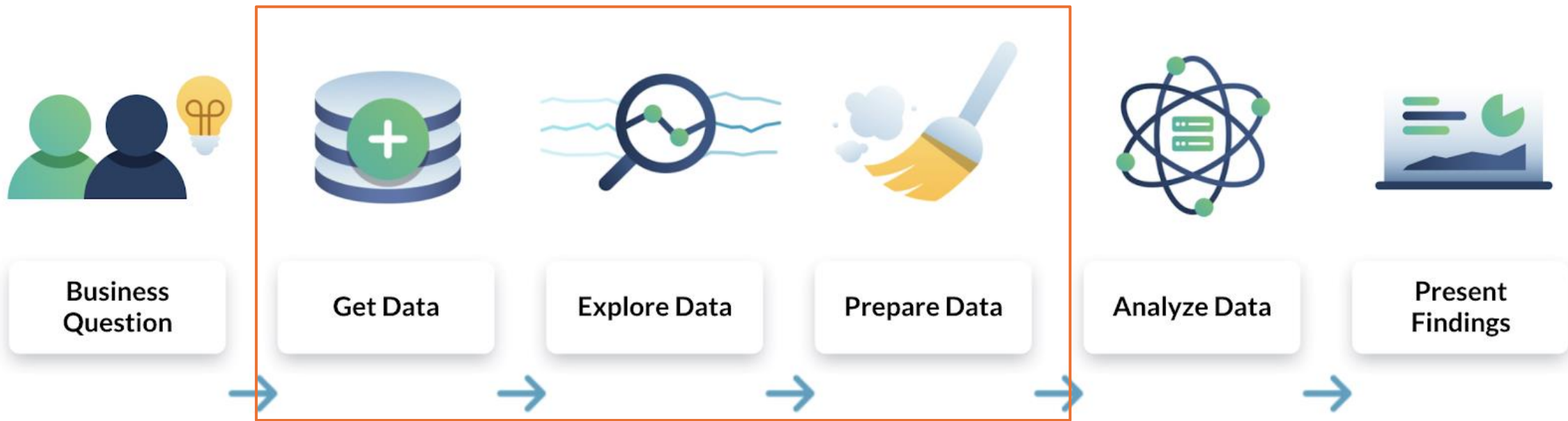
RANGOS

¿Hay valores imposibles según la regla de negocio?

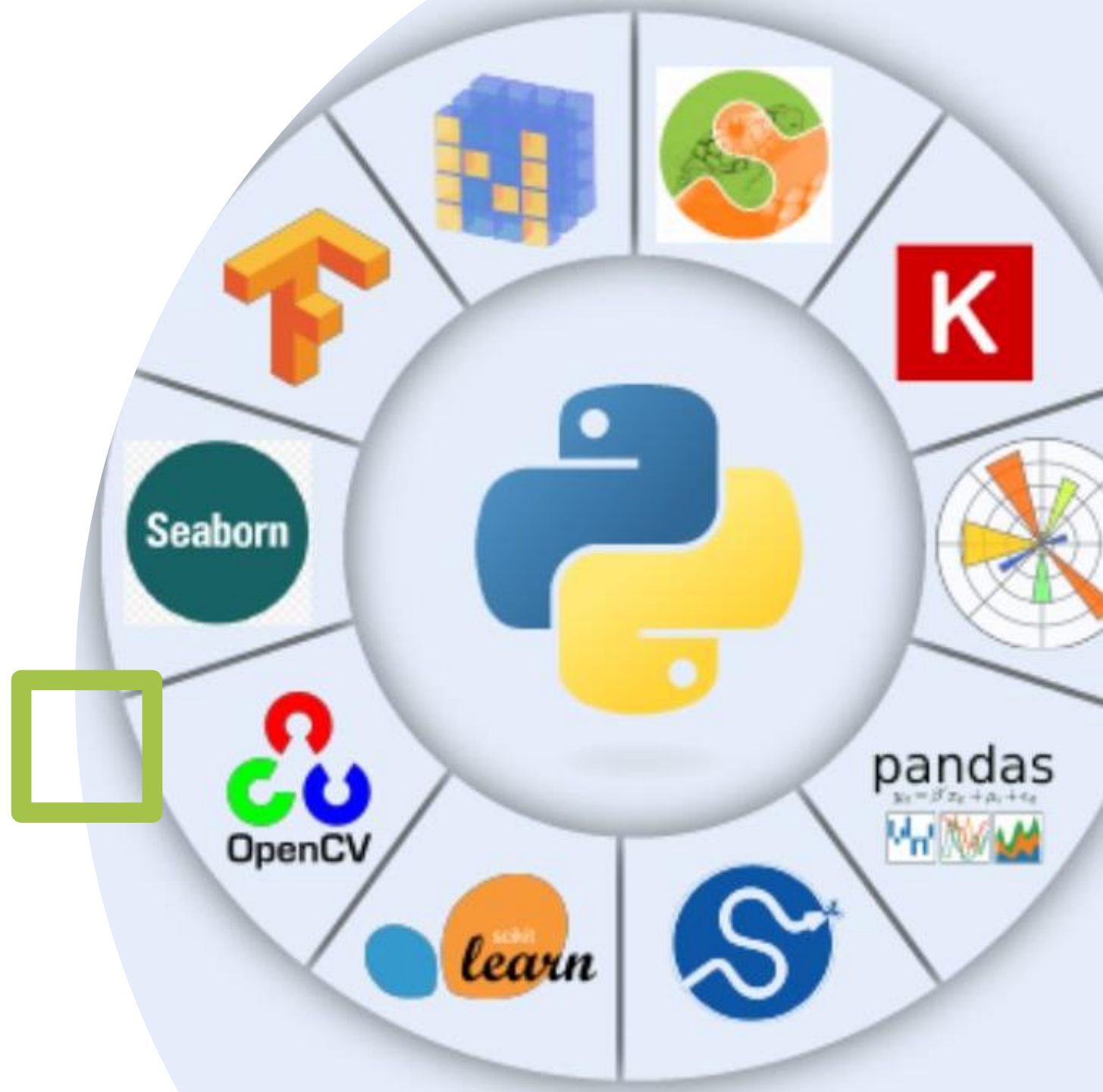
CATEGORÍAS

¿“web”, “WEB ” y “Web” significan lo mismo?

Evidencia → decisión documentada → verificación

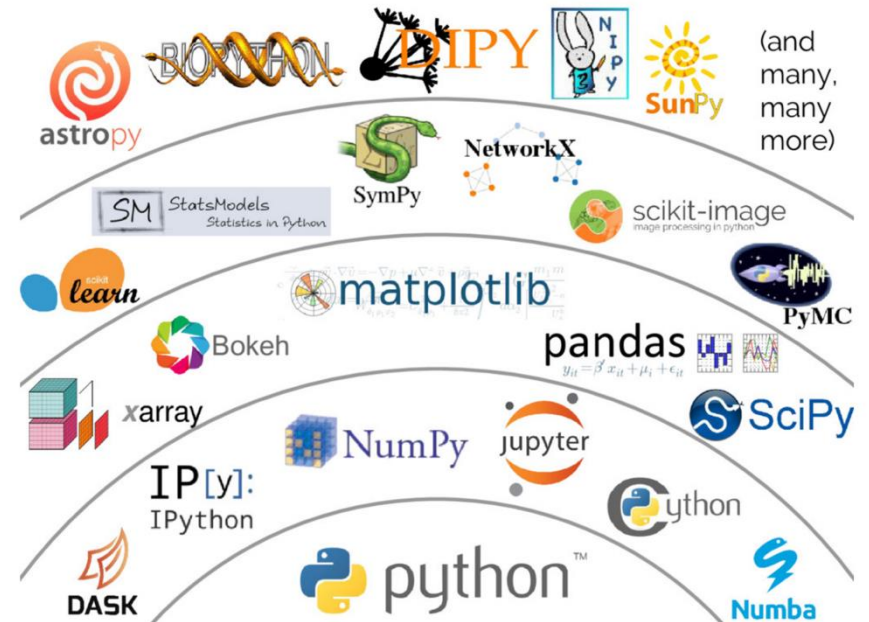


Librerías en Python



Librerías

- Las librerías (módulos) permiten **extender la funcionalidad** de Python.
- Un **módulo** es una porción de programa que realiza una operación específica.
- Una **librería** contiene un conjunto de **módulos**.



Librerías populares

NumPy

- Cálculos numéricos eficientes
- Arreglos y matrices multidimensionales

Pandas

- Análisis y manipulación de datos
- Estructuras de datos flexibles

Matplotlib

- Visualización de datos
- Gráficos en 2D

Scikit-Learn

- Aprendizaje automático
- Herramientas para análisis predictivo

TensorFlow

- Aprendizaje profundo
- Desarrollo de modelos de inteligencia artificial

Requests

- Peticiones HTTP sencillas
- Interacción con APIs web

Beautiful Soup

- Web scraping
- Extracción de datos de HTML y XML

Django

- Desarrollo web
- Framework de alto nivel

Flask

- Microframework web
- Desarrollo rápido y sencillo

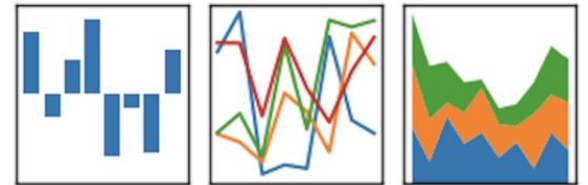
PyTorch

- Aprendizaje profundo
- Desarrollo de modelos de inteligencia artificial

Pandas

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



Pandas



Data frames

Es una colección de datos habitualmente **tabulada**.

Contenida en una **única tabla** cada **columna** representa una variable y cada **fila** representa un registro.

The diagram illustrates a data frame table with annotations. A blue bracket on the left points to the index labels (0-9). A red bracket at the top points to the column names. An orange bracket on the right points to the data cells. The table itself is as follows:

	Mountain	Height (m)	Range	Coordinates	Parent mountain	First ascent	Ascents bef. 2004	Failed attempts bef. 2004
0	Mount Everest / Sagarmatha / Chomolungma	8848	Mahalangur Himalaya	27°59'17"N 86°55'31"E	NaN	1953	>>145	121.0
1	K2 / Qogir / Godwin Austen	8611	Baltoro Karakoram	35°52'53"N 76°30'48"E	Mount Everest	1954	45	44.0
2	Kangchenjunga	8586	Kangchenjunga Himalaya	27°42'12"N 88°08'51"E	Mount Everest	1955	38	24.0
3	Lhotse	8516	Mahalangur Himalaya	27°57'42"N 86°55'59"E	Mount Everest	1956	26	26.0
4	Makalu	8485	Mahalangur Himalaya	27°53'23"N 87°05'20"E	Mount Everest	1955	45	52.0
5	Cho Oyu	8188	Mahalangur Himalaya	28°05'39"N 86°39'39"E	Mount Everest	1954	79	28.0
6	Dhaulagiri I	8167	Dhaulagiri Himalaya	28°41'48"N 83°29'35"E	K2	1960	51	39.0
7	Manaslu	8163	Manaslu Himalaya	28°33'00"N 84°33'35"E	Cho Oyu	1956	49	45.0
8	Nanga Parbat	8126	Nanga Parbat Himalaya	35°14'14"N 74°35'21"E	Dhaulagiri	1953	52	67.0
9	Annapurna I	8091	Annapurna Himalaya	28°35'44"N 83°49'13"E	Cho Oyu	1950	36	47.0

Lectura de datos con Pandas

```
In [ ]: #Leer archivo CSV
df = pd.read_csv("https://raw.githubusercontent.com/ulises1229/INTRO-PYTHON-
              ENESJ/master/data/salaries.csv")
```

Nota: El comando anterior tiene muchos argumentos opcionales para ajustar el proceso de importación de datos.

Hay una serie de comandos en pandas para leer **otros formatos de datos**:

```
pd.read_excel('myfile.xlsx', sheet_name='Sheet1', index_col=None, na_values=['NA'])
pd.read_stata('myfile.dta')
pd.read_sas('myfile.sas7bdat')
pd.read_hdf('myfile.h5', 'df')
```

Otros tipos de formatos para Big Data



JSON



CSV



Parquet



Avro



ORC



Explorar data frames

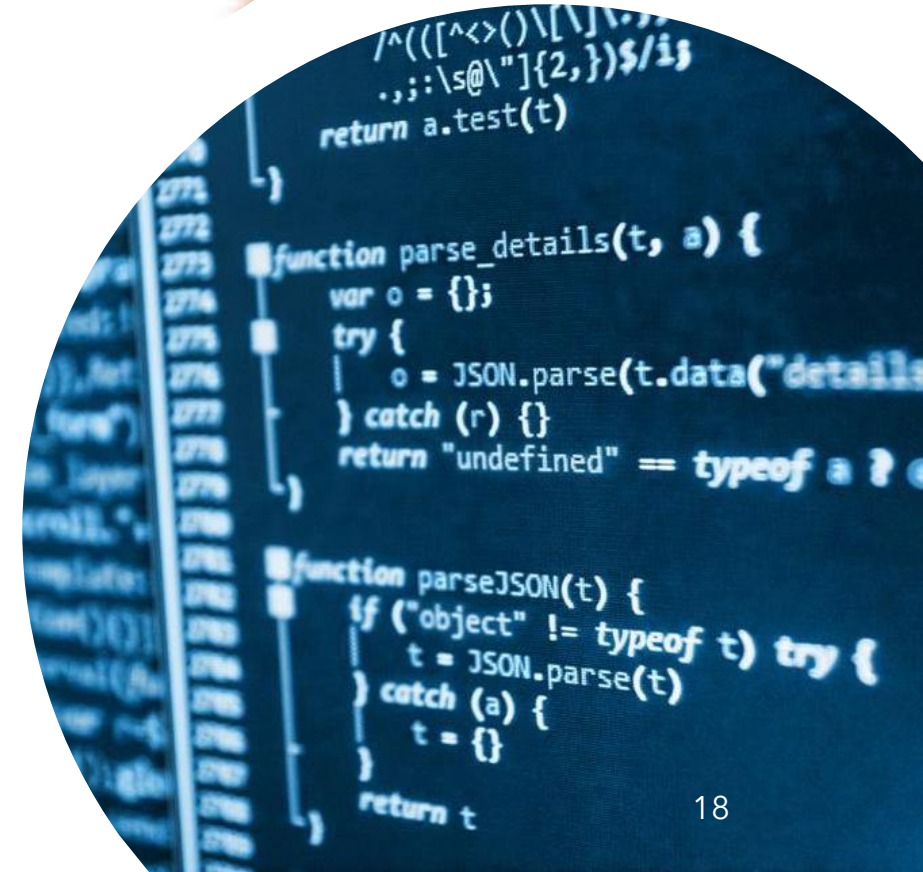
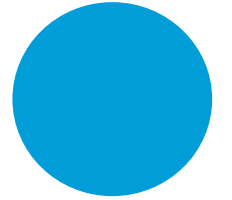
```
In [3]: #Lista de los primeros 5 registros  
df.head()
```

Out[3]:

	rank	discipline	phd	service	sex	salary
0	Prof	B	56	49	Male	186960
1	Prof	A	12	6	Male	93000
2	Prof	A	23	20	Male	110515
3	Prof	A	40	31	Male	131205
4	Prof	B	20	18	Male	104800

Ejercicio

- Se deberán leer los primeros 10, 20, 50 registros.
- ¿Podrían adivinar cómo ver los últimos registros?
- Pista:



Data Frames: Tipos de datos

Tipos en pandas	Tipos nativos en Python	Descripción
object	string	El dtype más general. Se asignará a la columna si la columna tiene tipos mixtos (números y cadenas).
int64	int	Caracteres numéricos. 64 se refiere a la memoria asignada para contener este carácter.
float64	float	Caracteres numéricos con decimales. Si una columna contiene números y NaNs (ver abajo), pandas usará por defecto float64, en caso de que el valor que falta tenga un decimal.
datetime64, timedelta[ns]	N/A (Ver el módulo datetime en Python)	Valores destinados a contener datos de tiempo. Busque en estos experimentos de series temporales.

Data Frame: Tipos de datos

```
In [4]: # Comprobar un tipo de columna determinado  
df['salary'].dtype
```

```
Out[4]: dtype('int64')
```

```
In [5]: #Comprueba los tipos de todas las columnas  
df.dtypes
```

```
Out[4]: rank      object  
         discipline object  
         phd       int64  
         service   int64  
         sex       object  
         salary    int64  
         dtype: object
```

Atributos de Data Frames

Los objetos de Python tienen atributos y métodos.

Atributos: usualmente se acceden a través de `.`

Métodos: usualmente se acceden a través de `()`

df.attribute	Descripción
dtypes	enumerar los tipos de las columnas
columns	listar los nombres de columna
axes	listar las etiquetas de fila y los nombres de columna
ndim	número de dimensiones
size	número de elementos
shape	devolver una tupla que representa la dimensionalidad
values	representación numpy de los datos

Ejercicio

1. Encuentre cuántos registros tiene este DataFrame
2. ¿Cuáles son los nombres de las columnas?
3. ¿Qué tipos de columnas tenemos en este data frame?

Métodos para Data Frames

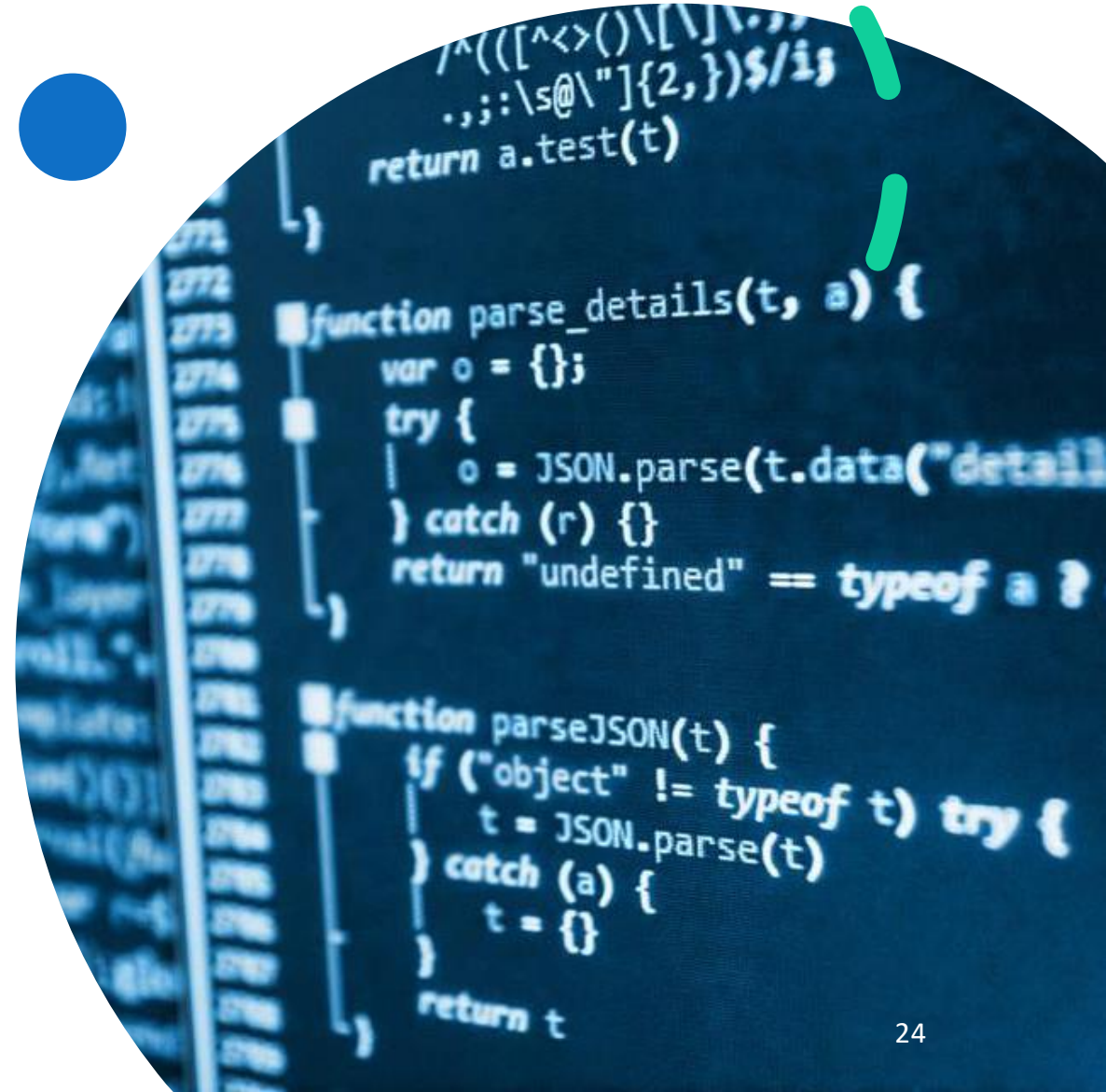
A diferencia de los atributos, los métodos en Python tienen paréntesis.

Todos los atributos y métodos se pueden enumerar con una función `dir()`: `dir(df)`

df.method()	Descripción
<code>head([n]), tail([n])</code>	Primera/última n filas
<code>describe()</code>	Generar estadísticas descriptivas (solo para columnas numéricas)
<code>max(), min()</code>	Devolver valores máximos/min para todas las columnas numéricas
<code>mean(), median()</code>	Devolver la media/mediana para todas las columnas numéricas
<code>std()</code>	Desviación estándar
<code>sample([n])</code>	devuelve una muestra aleatoria del marco de datos
<code>dropna()</code>	Eliminar todos los registros con valores NA

Ejercicio

1. Proporcione el resumen de las columnas numéricas del conjunto de datos
2. Calcular la desviación estándar para todas las columnas numéricas;
3. ¿Cuáles son las medias de los primeros 50 registros del conjunto de datos?
 - **Sugerencia:** Utilice el método `head()` para crear un subconjunto de los primeros 50 registros y, a continuación, calcule la media



Selección de una columna en el Data Frame

Método 1: Subconjunto del DF utilizando el nombre de columna:

`df['sex']`

Método 2: Utilice el nombre de columna como atributo:

`df.sex`

Nota: Existe un atributo `rank` para los data frames de pandas, por lo que para seleccionar una columna con un nombre "rank" debemos usar el método 1.

Ejercicio

1. Calcular las estadísticas básicas para la columna de salario;
2. Encuentre cuántos valores hay en la columna de salario (utilice el método count);
3. Calcular la media para la variable salario;

```
return a.test(t)

function parse_details(t, a) {
  var o = {};
  try {
    o = JSON.parse(t.data("detail"))
  } catch (r) {}
  return "undefined" == typeof a ?

function parseJSON(t) {
  if ("object" != typeof t) try {
    t = JSON.parse(t)
  } catch (a) {
    t = {}
  }
  return t
}
```

Método *groupby* para Data Frames

Usando el método "**group by**" podemos:

- Dividir los datos en grupos en función de algunos criterios
- Calcular estadísticas (o aplicar una función) a cada grupo
- Similar a la función `dplyr()` en R

```
In [ ]: #Agrupando los datos usando rank  
df_rank = df.groupby(['rank'])
```

```
In [ ]: #Calcular la media de cada  
columna numérica por cada grupo  
df_rank.mean()
```

	phd	service	salary
rank			
AssocProf	15.076923	11.307692	91786.230769
AsstProf	5.052632	2.210526	81362.789474
Prof	27.065217	21.413043	123624.804348

Método *groupby* para Data Frames

Una vez creado el objeto *groupby* podemos calcular varias estadísticas para cada grupo:

```
In [ ]: #Calculate mean salary for each professor rank:  
df.groupby('rank')[['salary']].mean()
```

salary	
rank	
AssocProf	91786.230769
AsstProf	81362.789474
Prof	123624.804348

Nota: Si se utilizan corchetes individuales para especificar la columna (por ejemplo, salario), la salida es el objeto Pandas Series. Cuando se utilizan corchetes dobles, la salida es un data frame

Método *groupby* para Data Frames

- **groupby** notas de rendimiento:
 - No se produce ninguna agrupación/división hasta que sea necesario.
 - La creación del objeto *groupby* solo comprueba que ha pasado una asignación válida
 - Por defecto las claves de grupo se ordenan durante la operación *groupby*. Se puede que desee pasar *sort=False* para una posible aceleración:

```
In [ ]: #Calculate mean salary for each professor rank:  
df.groupby(['rank'], sort=False)[['salary']].mean()
```

Data Frame: filtering

Para crear un subconjunto de los datos, podemos aplicar la indexación booleana. Esta indexación se conoce comúnmente como filtro. Por ejemplo, si queremos crear un subconjunto de las filas en las que el valor salarial es mayor que \$120K:

```
In [ ]: #Calcular el salario promedio para cada rango de profesor:  
df_sub = df[ df['salary'] > 120000 ]
```

Cualquier operador booleano se puede utilizar para crear un subconjunto de los datos:

```
In [ ]: #Select only those rows that contain female professors:  
df_f = df[ df['sex'] == 'Female' ]
```

Data Frames: Ordenamientos

Podemos ordenar los datos por un valor en la columna. De forma predeterminada, la ordenación se producirá en orden ascendente y se devolverá un nuevo marco de datos.

```
In [ ]: # Cree un nuevo marco de datos a partir del original ordenado por la
        columna Salario
        df_sorted = df.sort_values( by ='service')
        df_sorted.head()
```

Out[]:

	rank	discipline	phd	service	sex	salary
55	AsstProf	A	2	0	Female	72500
23	AsstProf	A	2	0	Male	85000
43	AsstProf	B	5	0	Female	77000
17	AsstProf	B	4	0	Male	92000
12	AsstProf	B	1	0	Male	88000

Data Frames: Ordenamientos

Podemos ordenar los datos usando 2 o más columnas:

```
In [ ]: df_sorted = df.sort_values( by=['service', 'salary'], ascending = [True, False])
df_sorted.head(10)
```

Out[]:

	rank	discipline	phd	service	sex	salary
52	Prof	A	12	0	Female	105000
17	AsstProf	B	4	0	Male	92000
12	AsstProf	B	1	0	Male	88000
23	AsstProf	A	2	0	Male	85000
43	AsstProf	B	5	0	Female	77000
55	AsstProf	A	2	0	Female	72500
57	AsstProf	A	3	1	Female	72500
28	AsstProf	B	7	2	Male	91300
42	AsstProf	B	4	2	Female	80225
68	AsstProf	A	4	2	Female	77500

Ejercicio

1. Se deberán recuperar el profesor **mejor y peor pagado** de todo el dataframe.
2. Se deberá realizar el **ejercicio anterior por sexo**.
3. Se deberá calcular un **promedio del salario** por sexo.



Valores faltantes

Los valores que faltan se marcan como NaN

```
In [ ]: # Leer un conjunto de datos con valores faltantes
vuelos = pd.read_csv(" https://raw.githubusercontent.com/ulises1229/INTRO-PYTHON-ENESJ/master/data/flights.csv ")
```

```
In [ ]: # Seleccione las filas que tienen al menos un valor faltante
vuelos[vuelos.isnull().any(axis=1)].head()
```

```
Out[ ]:
```

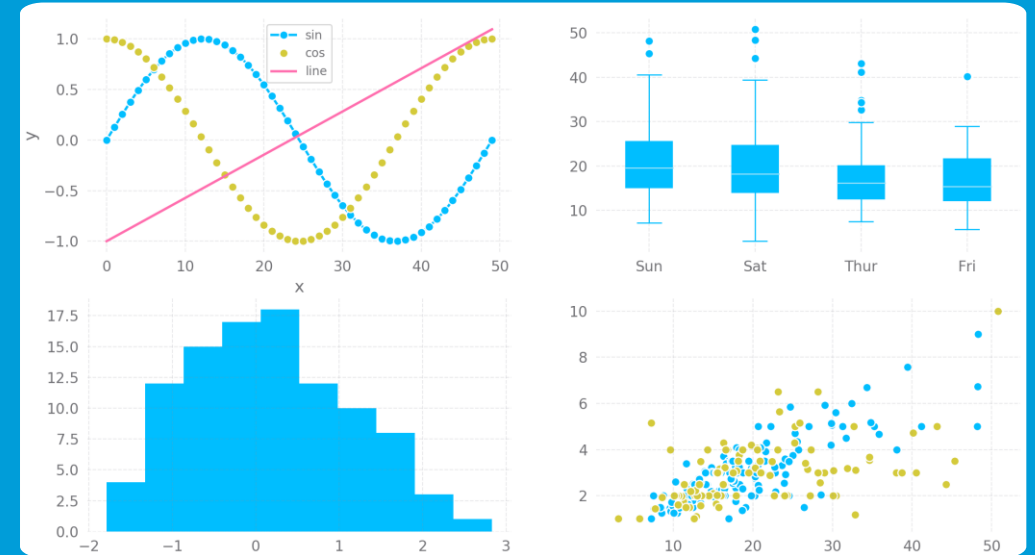
	year	month	day	dep_time	dep_delay	arr_time	arr_delay	carrier	tailnum	flight	origin	dest	air_time	distance	hour	minute
330	2013	1	1	1807.0	29.0	2251.0	NaN	UA	N31412	1228	EWB	SAN	NaN	2425	18.0	7.0
403	2013	1	1	NaN	NaN	NaN	NaN	AA	N3EHAA	791	LGA	DFW	NaN	1389	NaN	NaN
404	2013	1	1	NaN	NaN	NaN	NaN	AA	N3EVAA	1925	LGA	MIA	NaN	1096	NaN	NaN
855	2013	1	2	2145.0	16.0	NaN	NaN	UA	N12221	1299	EWB	RSW	NaN	1068	21.0	45.0
858	2013	1	2	NaN	NaN	NaN	NaN	AA	NaN	133	JFK	LAX	NaN	2475	NaN	NaN

Valores faltantes

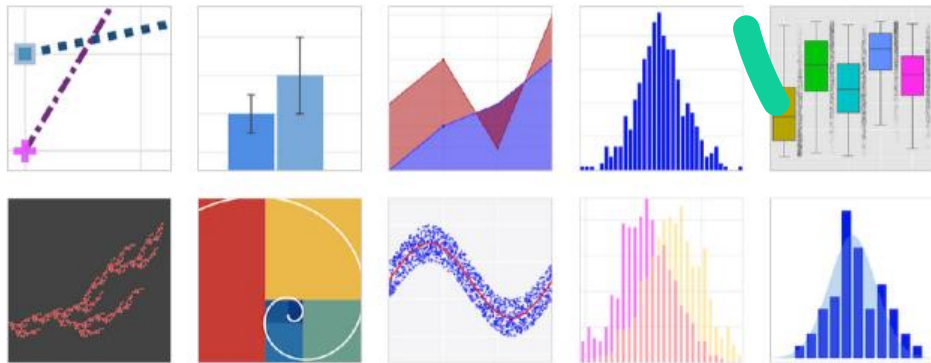
Hay una serie de métodos para tratar con los valores que faltan en el marco de datos:

df.method()	Descripción
dropna()	Retirar observaciones faltantes
dropna(how='all')	Elimina todas las celdas con NA
dropna(axis=1, how='all')	Elimina la columna si faltan todos los valores
dropna(thresh = 5)	Elimina las filas que contienen menos de 5 valores que no faltan
fillna(0)	Reemplaza los valores faltantes por ceros
isnull()	Devuelve True si falta el valor
notnull()	Devuelve True para los valores que no faltan

Visualización de datos con

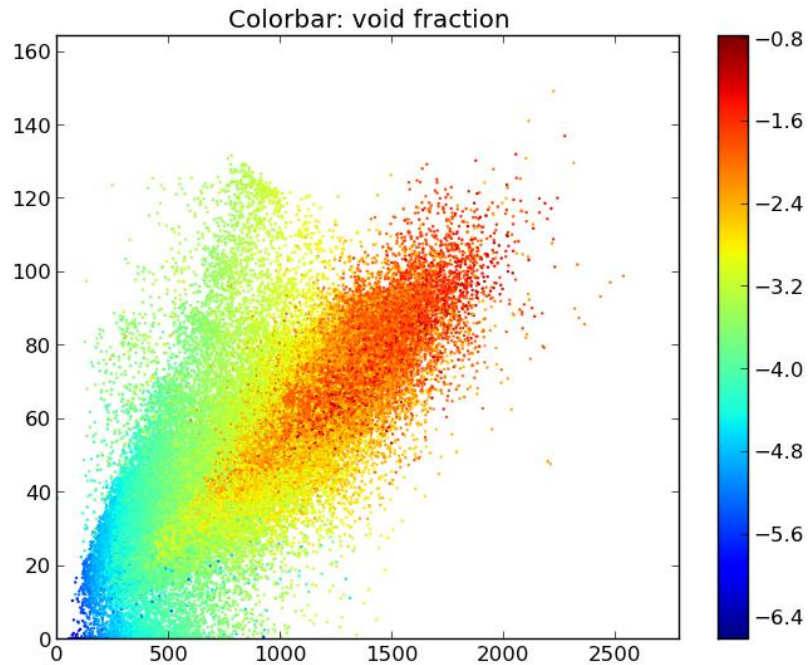


Tipos de gráficos



El tipo de gráfico es un aspecto muy importante, debido a que **es el mensaje que se proyectará**. Entonces, se deber seleccionar el gráfico de acuerdo a los datos con los que se cuenta .

Gráficos de dispersión (scatter)



- Mostrar variables cuantitativas en un intervalo específico.
- Los puntos se deberán distinguir claramente.
- Si existe más de una categoría de deberá utilizar colores y formas para hacer la distinción entre ambas categorías.

Diagrama Dispersión

Utilizar dataframe `salary.csv` para generar un gráfico de dispersión.

```
# importar la librería de pandas como pd
import pandas as pd

#Leer el dataframe salaries.csv
df = pd.read_csv("https://raw.githubusercontent.com/ulises1229/INTRO-PYTHON-ENESJ/master/data/salaries.csv")
df.head()
```

Diagrama Dispersión

```
# Almacenar variables años y salario
x = df.service
y = df.salary

# Importar librería Matplotlib
import matplotlib.pyplot as plt

# Generar diagrama de dispersión
plt.scatter(x, y)

# Títulos: gráfico y ejes
plt.title("Gráfico de Dispersión Años de servicio vs Salario")
plt.xlabel('Años de servicio')
plt.ylabel('Salario Anual')

# Desplegar imagen en pantalla
plt.show()
```

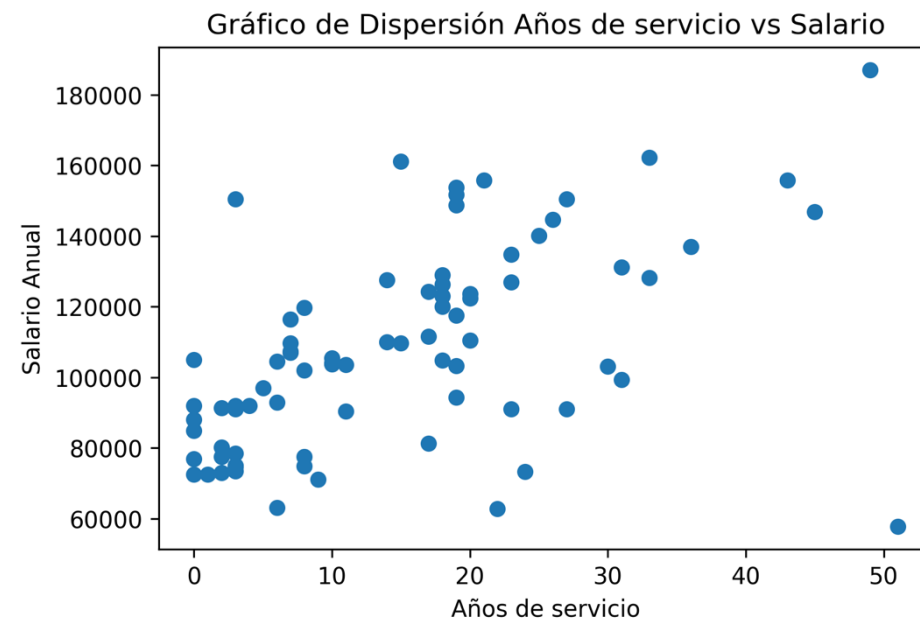


Diagrama Dispersión

- Agregar colores y tamaño.

```
# Importar librería de numpy
import numpy as np

# Almacenar variables x = años servicio y y = salario
x = df.service
y = df.salary

# Arreglo de colores
colores = np.arange(len(x))
print(colores)

# Agregar un scatter con colores y transparencia
plt.scatter(x, y, c=colores, alpha = 0.5, s = (y*0.002))
```

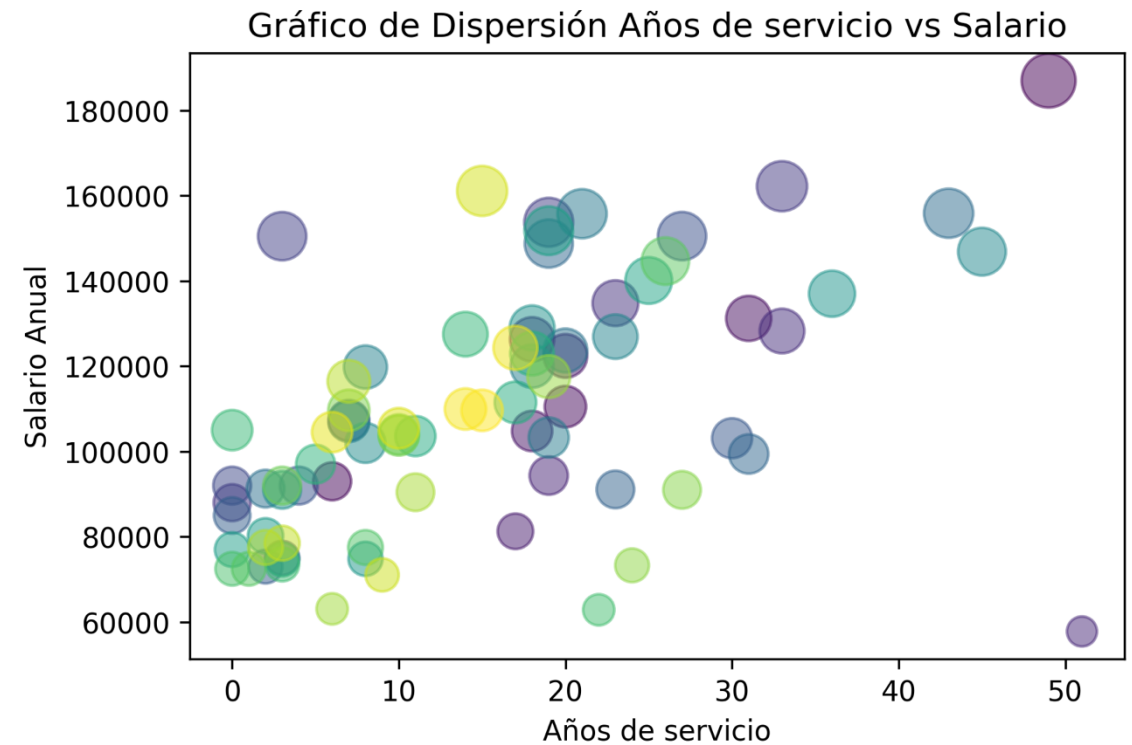


Diagrama Dispersión

- Agregar colores, tamaño y leyenda.

```
# Almacenar variables x = años servicio y y = salario
x = df.service
y = df.salary
lab = ['Male', 'Female']

# Arreglo de colores
colores = np.arange(len(x))

# Títulos: gráfico y ejes
plt.title("Gráfico de Dispersión Años de servicio vs Salario")
plt.xlabel('Años de servicio')
plt.ylabel('Salario Anual')

# Agregar un scatter con colores, transparencia y leyenda
plt.scatter(x, y, c=colores, alpha = 0.5, s = (y*0.002), label = "datos")
plt.legend(loc='best')

plt.show()
```

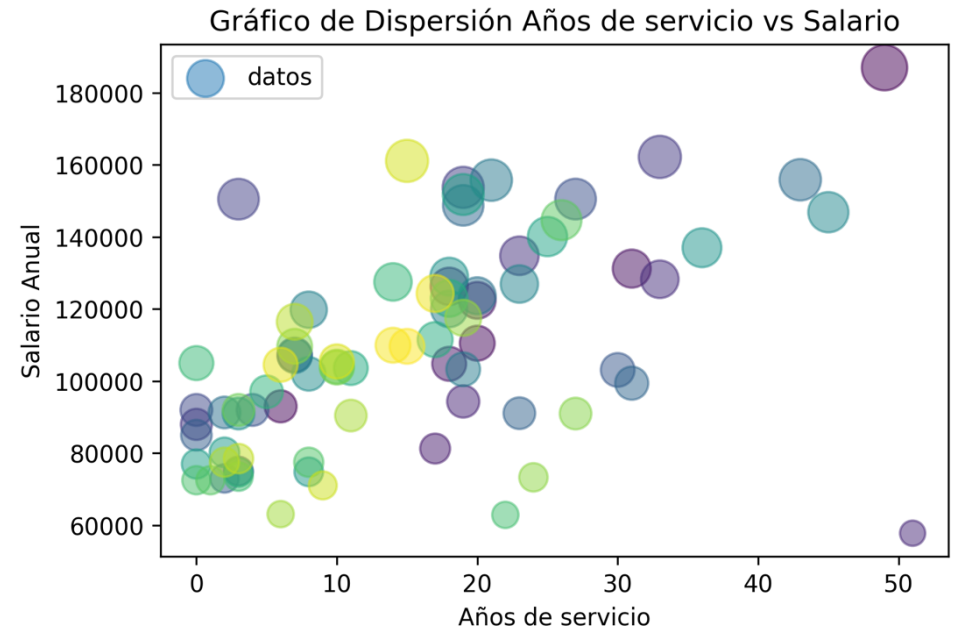
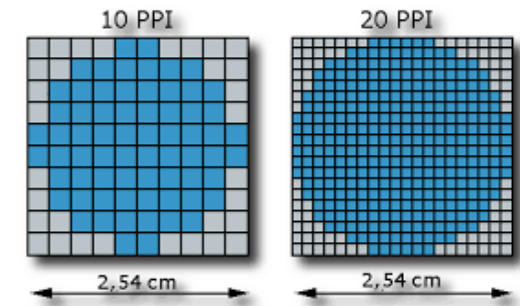
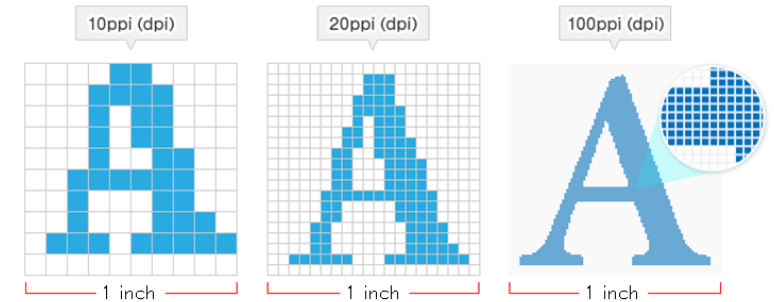


Diagrama de Dispersión – Exportando un Gráfico

Con matplotlib es posible exportar imágenes de alta calidad. Para esto es necesario definir elementos tales como:

- Título de gráfico
- Títulos de los ejes
- Título de la leyenda

```
# guarda la imagen en un archivo  
plt.savefig("dispersión.tiff", dpi=300, quality = 95, bbox_inches='tight')
```



MPG dataframe

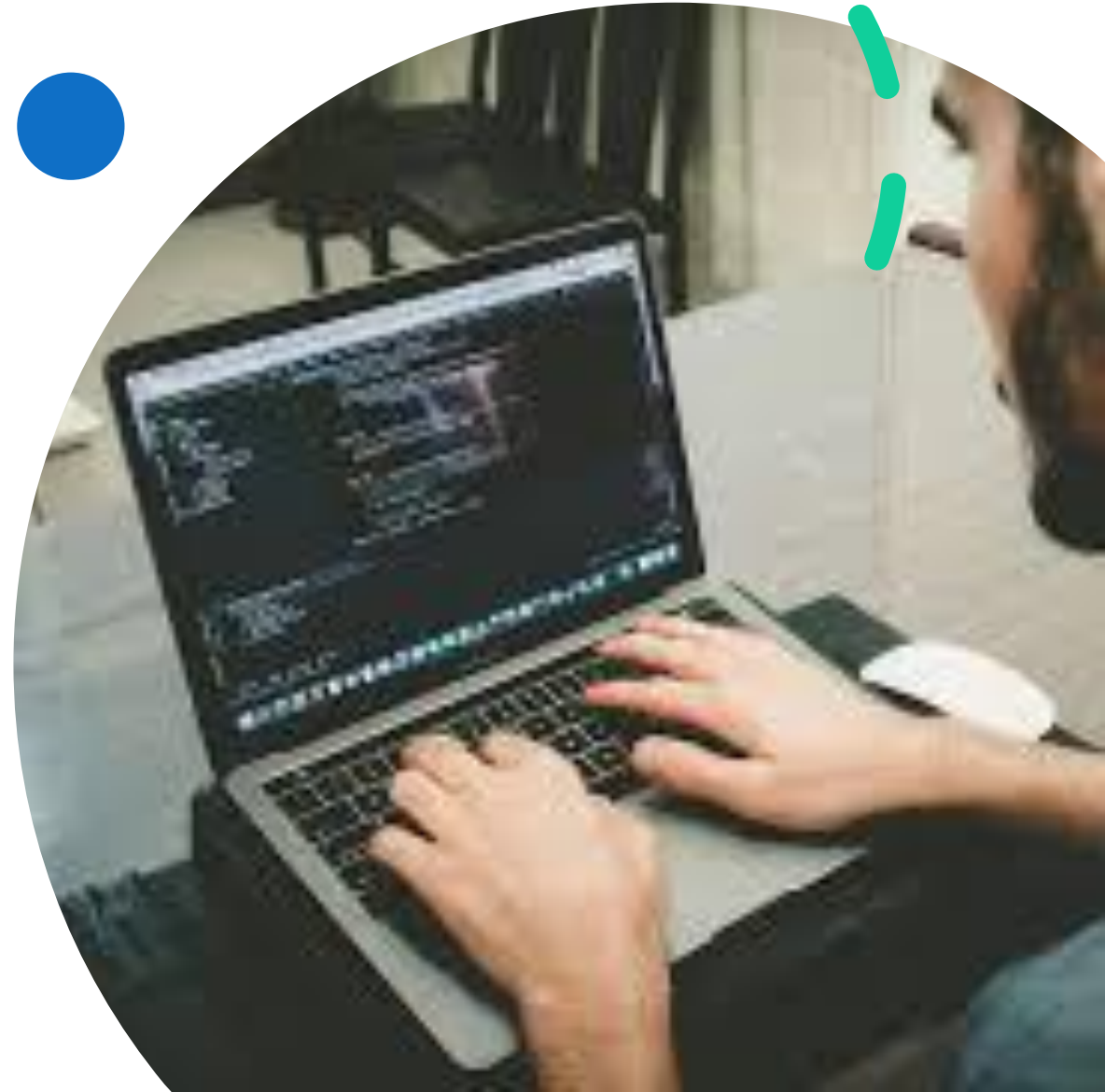
El data frame cuenta con 234 filas, 38 modelos de automóviles (1999 - 2008)
11 columnas (variables).

	manufacturer	model	displ	year	cyl	trans	drv	cty	hwy	fl	class
1	audi	a4	1.8	1999	4	auto(l5)	f	18	29	p	compact
2	audi	a4	1.8	1999	4	manual(m5)	f	21	29	p	compact
3	audi	a4	2.0	2008	4	manual(m6)	f	20	31	p	compact
4	audi	a4	2.0	2008	4	auto(av)	f	21	30	p	compact
5	audi	a4	2.8	1999	6	auto(l5)	f	16	26	p	compact
6	audi	a4	2.8	1999	6	manual(m5)	f	18	26	p	compact
7	audi	a4	3.1	2008	6	auto(av)	f	18	27	p	compact
8	audi	a4 quattro	1.8	1999	4	manual(m5)	4	18	26	p	compact
9	audi	a4 quattro	1.8	1999	4	auto(l5)	4	16	25	p	compact
10	audi	a4 quattro	2.0	2008	4	manual(m6)	4	20	28	p	compact
11	audi	a4 quattro	2.0	2008	4	auto(s6)	4	19	27	p	compact
12	audi	a4 quattro	2.8	1999	6	auto(l5)	4	15	25	p	compact
13	audi	a4 quattro	2.8	1999	6	manual(m5)	4	17	25	p	compact
14	audi	a4 quattro	3.1	2008	6	auto(s6)	4	17	25	p	compact
15	audi	a4 quattro	3.1	2008	6	manual(m6)	4	15	25	p	compact



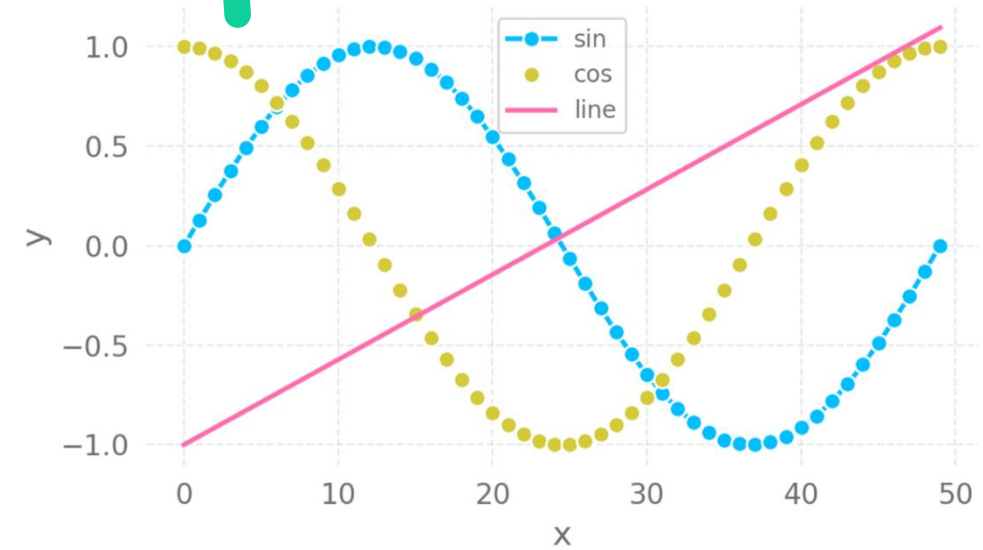
Ejercicio

- Emplear el dataframe mpg.csv
- Se deberá generar un gráfico de dispersión del desplazamiento del motor vs rendimiento en autopista.
 1. Se deberá generar un segundo gráfico de desplazamiento del motor vs rendimiento en ciudad.
 2. Exportar un gráfico con una resolución de 300 dpi



Gráficos de Línea

- Representar valores cuantitativos en función de la variable independiente.
- Es posible combinar la gráfica con puntos, para enfatizar valores específicos.
- Si existe más de una categoría en la gráfica, deberán utilizarse leyendas y tipos de línea para diferenciar los datos.



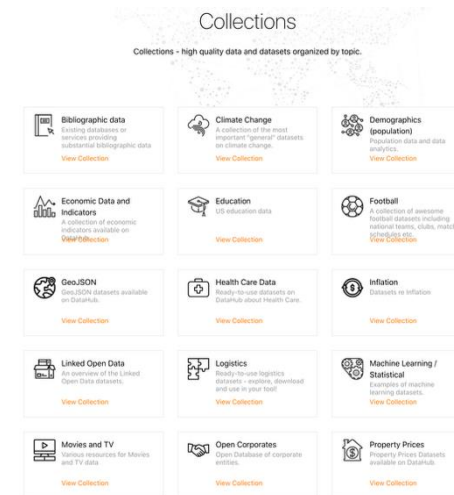
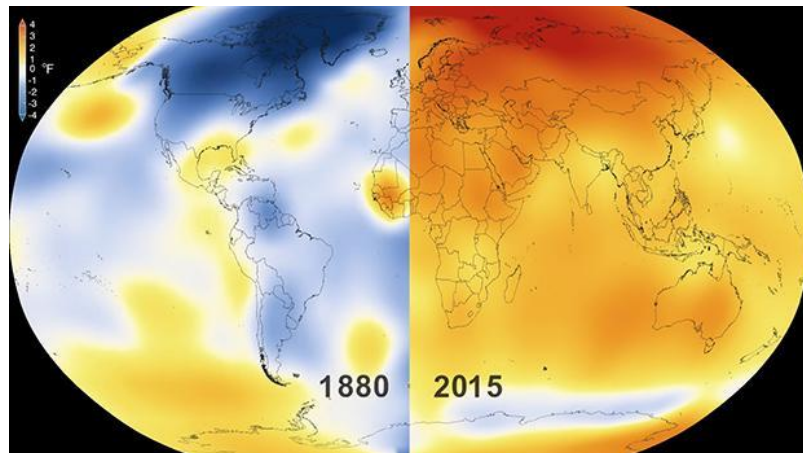
Gráficos de línea

- Se deberá importar el dataframe temp.csv (Repositorio GH)

```
# Importar pandas y matplotlib
import matplotlib.pyplot as plt
import pandas as pd

# Importar data frame de temperatura
df_temp = pd.read_csv('https://raw.githubusercontent.com/ulises1229/INTRO-PYTHON-ENESJ/master/data/temp.csv', sep=',')

df_temp
```



Gráficos de línea

```
# Importar pandas y matplotlib
import matplotlib.pyplot as plt
import pandas as pd

# Importar data frame de temperatura
df_temp = pd.read_csv('https://raw.githubusercontent.com/ulises1229/INTRO-PYTHON-ENESJ/master/data/temp.csv', sep=',')

df_temp

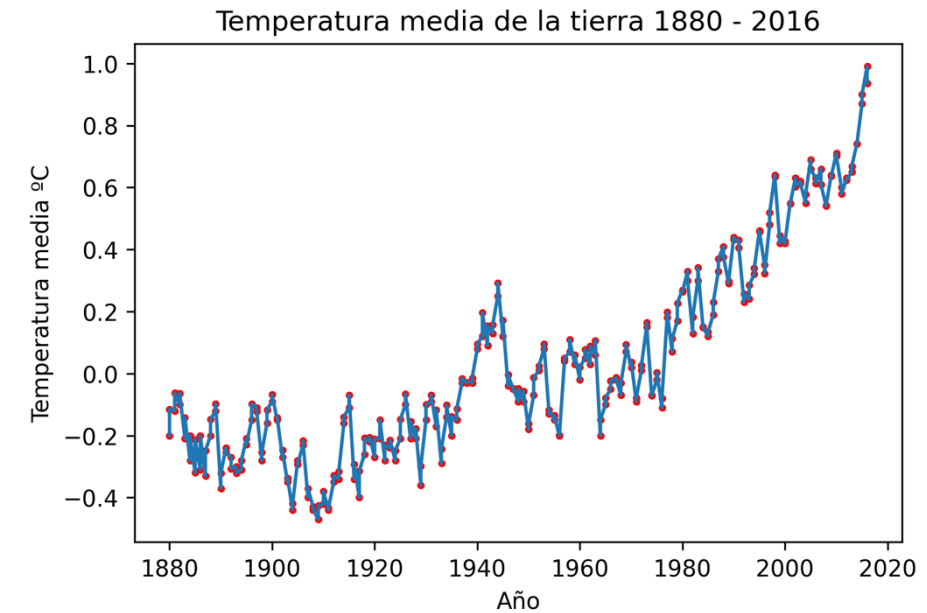
# Separa dos variables del dataframe
año = df_temp.Year
temp = df_temp.Mean

# Generar el diagrama de línea
plt.plot(año,temp)

#Agregar un scatter
plt.scatter(año, temp, s = 5, c = "red")

# Agregar títulos
plt.title("Temperatura media de la tierra 1880 - 2016")
plt.xlabel('Año')
plt.ylabel('Temperatura media °C')

#Guardar el gráfico en un archivo
plt.savefig("line_temp.tiff", dpi=300, quality = 95, bbox_inches='tight')
```



Ejercicio 3

Emplear el dataframe **co2.csv**

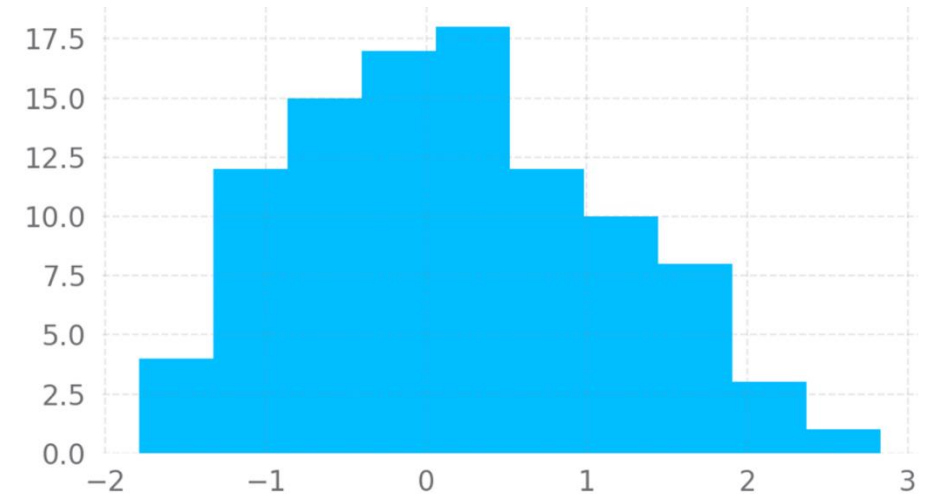
Está en el repositorio de **GitHub**

1. Se deberá generar un gráfico de línea con **fecha** vs **Interpolated**.
2. Se deberá generar un gráfico de línea con **fecha** vs **Trend**.
3. Se deberá exportar el gráfico con una calidad de 300 dpi.



Gráficos de barras

- Mostrar comparaciones, desviaciones o rankings. Puede estar en formato vertical u horizontal.
- Utilizar solo un color, a menos que haya más de una categoría. En dado caso se usan leyendas.
- Evitar utilizar bordes para las barras.
- Distancia entre barras, deberá ser suficiente para que no se traslapen.



Gráficos de barras

```
# Importar pandas y matplotlib
import matplotlib.pyplot as plt
import pandas as pd

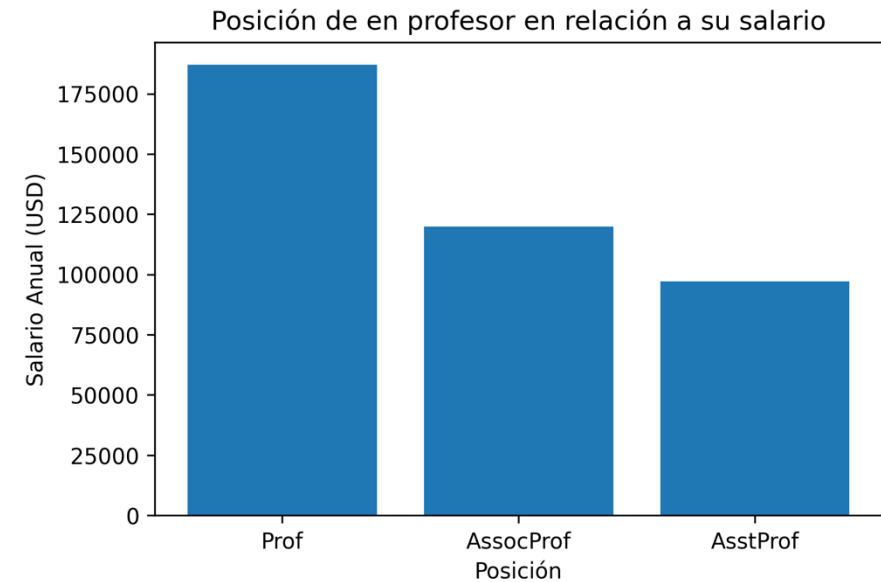
#Leer el dataframe salaries.csv
df = pd.read_csv("https://raw.githubusercontent.com/ulises1229/INTRO-PYTHON-ENESJ/master/data/salaries.csv")

# Recuperar rank y salario
rank = df['rank']
sueldo = df.salary

# Generar una gráfica de barras
plt.bar(rank, sueldo)

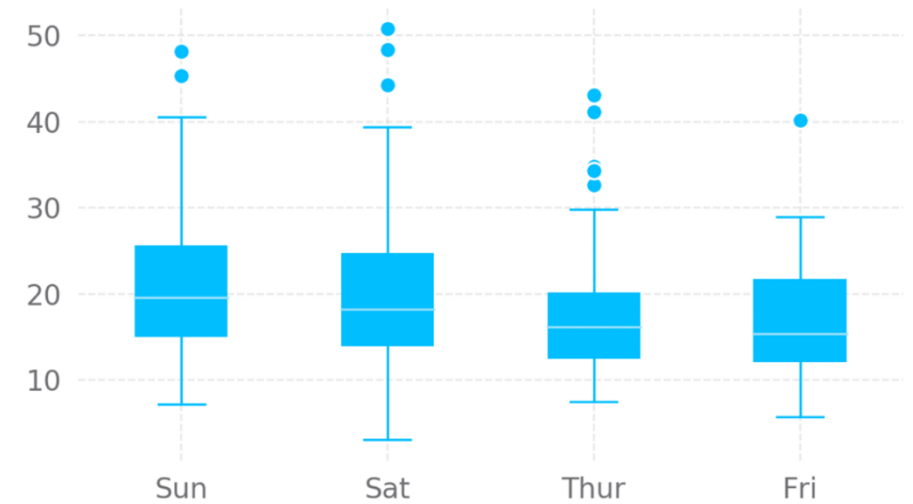
# Agregar títulos
plt.title("Posición de en profesor en relación a su salario")
plt.xlabel("Posición")
plt.ylabel("Salario Anual (USD)")

plt.savefig("salario.tiff", dpi=300, quality = 95, bbox_inches='tight')
```



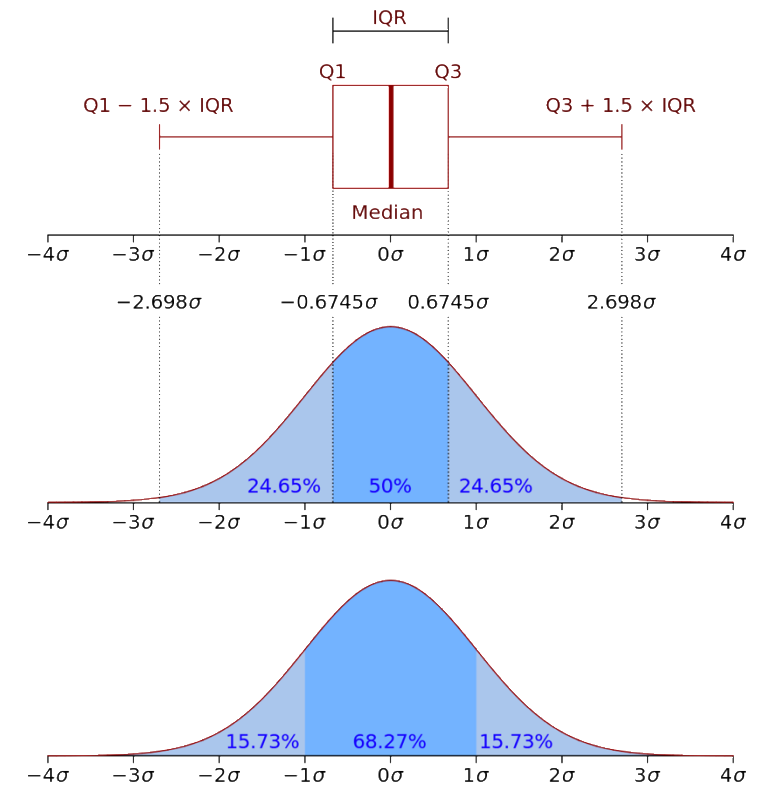
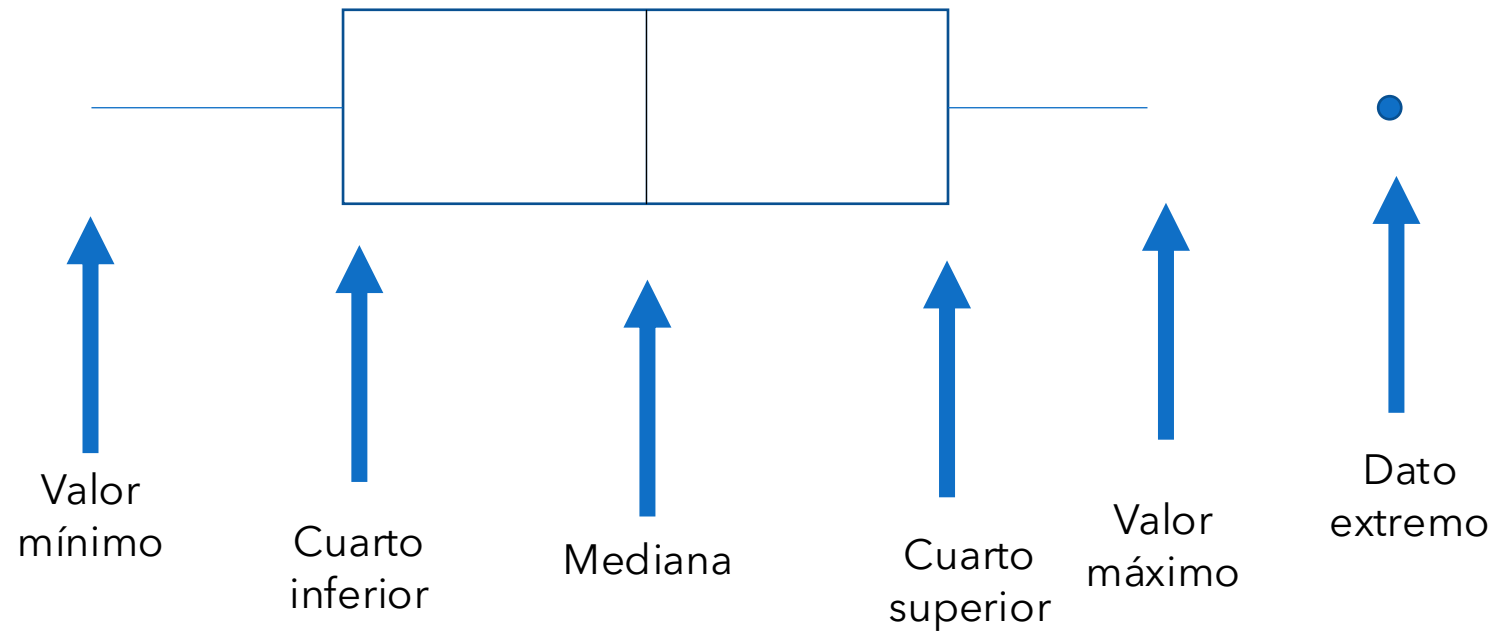
Boxplots

- y variable numérica
- x variable nominal
(categórica, objetos)



Boxplots

Cuartiles: Valores que dividen la muestra en 4 partes iguales



Boxplots

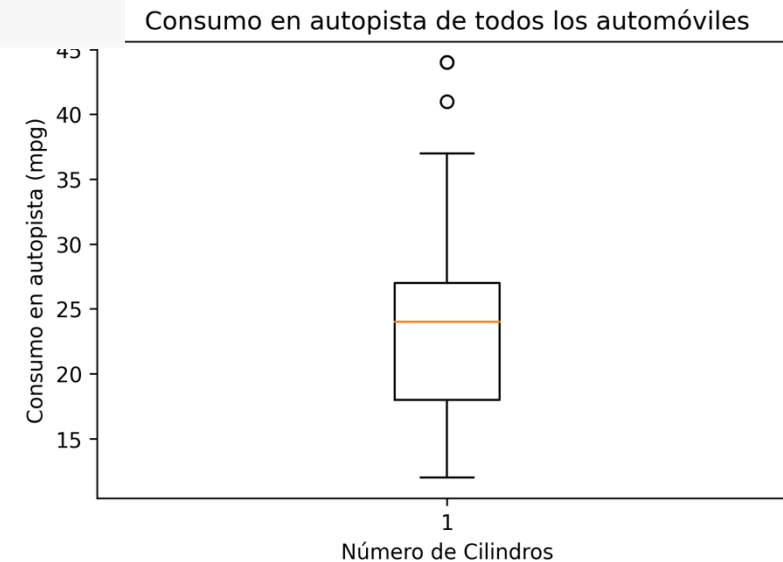
```
# Importar pandas y matplotlib
import matplotlib.pyplot as plt
import pandas as pd

# importar ma

#Leer el dataframe salaries.csv
mpg = pd.read_csv("https://raw.githubusercontent.com/ulises1229/INTRO-PYTHON-ENESJ/master/data/mpg.csv")

plt.boxplot(mpg.hwy)
# Agregar títulos
plt.title("Consumo en autopista de todos los automóviles")
plt.xlabel("Número de Cilindros")
plt.ylabel("Consumo en autopista (mpg)")

# exportar imagen
plt.savefig("mpg.tiff", dpi=300, quality = 95, bbox_inches='tight')
```



Análisis Exploratorio de Datos



AED: del dato real a la evidencia

Antes del modelo, el grupo debe defender qué se conserva, qué se transforma y qué riesgos quedan en un dataset real.



Mensaje didáctico

- Dataset UCI Bank Marketing: campañas telefónicas reales de una institución bancaria portuguesa.
- 45,211 registros; objetivo: suscribió depósito a plazo.
- El AED no es “hacer gráficas”: es reducir incertidumbre antes de decidir.